

Motivation

Large language models have become the dominant paradigm for a wide range of NLP tasks but full fine-tuning remains prohibitively expensive for researchers working under academic compute constraints. Parameter-efficient fine-tuning (PEFT) methods address this by updating only a small subset of parameters while keeping the base model frozen. However, the design space of PEFT is large: methods differ in the number of trainable parameters, where in the model they intervene, and their compatibility with low-precision training. Despite growing practical adoption, there exists no systematic characterization of how these methods trade off statistical performance against hardware efficiency under a fixed compute budget. This project fills that gap by empirically mapping the efficiency frontier of three representative PEFT methods (LoRA [1], IA³ [2], and prefix tuning [3]) across multiple downstream tasks and precision regimes. Rather than asking which method achieves the best accuracy in isolation, we seek to look for a configuration that maximizes downstream task performance given a fixed GPU memory budget or a fixed wall-clock budget.

Proposed System

We will implement a unified fine-tuning harness around Qwen2.5-1.5B using the Hugging Face PEFT and Transformers libraries. The harness will support swappable adapter backends (LoRA, IA³, prefix tuning) and configurable training precision (FP32, BF16, INT4 via QLoRA [4]). For LoRA specifically, we will ablate over rank values (r) ranging from 4 to 64. All configurations will be evaluated on three downstream tasks:

- SST-2 (sentiment classification)
- HellaSwag (commonsense reasoning)
- GSM8K (arithmetic reasoning)

For each configuration, we will measure two primary efficiency metrics: accuracy per trainable parameter (statistical efficiency) and accuracy per GPU-hour (hardware efficiency). Together these metrics provide practitioners with actionable guidance for choosing PEFT configurations under real academic compute constraints (single RTX PRO 6000 Blackwell GPU).

The baseline for all comparisons is full finetuning with BF16 precision when feasible. Where full fine-tuning exceeds available memory, we plan on using the highest-rank LoRA ($r=64$) in BF16 as the practical upper bound.

Preliminary Work

As preliminary exploration, we have verified environment feasibility by successfully loading Qwen2.5-1.5B in BF16 on a single RTX PRO 6000 Blackwell GPU (102 GB VRAM) and attaching LoRA adapters across ranks (r) using the Hugging Face PEFT library. Single training steps complete without error, with peak VRAM ranging from 6.80 GB ($r=4$) to 6.85 GB ($r=64$) — confirming that rank has negligible impact on memory at this model scale and leaving substantial headroom for full fine-tuning baselines (7.18 GB peak). We have also confirmed that INT4 quantization via bitsandbytes NF4 is supported: QLoRA ($r=16$, INT4 base + BF16 adapters) peaks at just 1.79 GB VRAM, a 74% reduction relative to BF16 LoRA. This establishes the core technical stack is functional and the full experiment matrix is feasible within our compute constraints.

References

- [1] Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022. [covered in course]
- [2] Liu et al. Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning. NeurIPS 2022.
- [3] Li and Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. ACL 2021.
- [4] Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023.

Tentative Experiment Plans

Experiment Plan 1 — LoRA Rank vs. Task Performance (Member 1)

We hypothesize that increasing LoRA rank should improve downstream accuracy up to a task-dependent saturation point, after which gains should diminish while memory usage and training time continue to grow. Simpler tasks like SST-2 should saturate at lower ranks than more complex reasoning tasks such as GSM8K. To evaluate this, we measure task accuracy alongside hardware costs (peak GPU memory and training time) and derive efficiency metrics such as accuracy per trainable parameter. We plan on finetuning Qwen2.5-1.5B with LoRA across ranks on SST-2, HellaSwag, and GSM8K, keeping all other hyperparameters fixed and repeat each configuration with three random seeds to estimate variability.

Experiment Plan 2 — Precision Regime and QLoRA (Member 2)

We hypothesize that INT4 quantization of the base model (QLoRA) will substantially reduce peak VRAM while incurring only minor accuracy degradation, since the BF16 adapter weights still capture most task-specific updates. To test this, we plan on comparing downstream performance on specific tasks alongside hardware metrics such as peak memory usage and training throughput, and derive efficiency measures like accuracy per GPU-hour. We will fix LoRA rank at $r=16$ and fine-tune Qwen2.5-1.5B under three precision regimes (FP32, BF16, and INT4) with BF16 adapters (QLoRA).

Experiment Plan 3 — PEFT Method Comparison (Member 3)

We hypothesize that LoRA will achieve the highest accuracy per trainable parameter across tasks due to its more expressive low-rank weight updates, while IA³ will be the most hardware-efficient because of its minimal parameter footprint; prefix tuning is expected to fall in between, offering moderate expressivity without directly modifying weights. To test this, we plan on comparing downstream performance alongside hardware metrics such as trainable parameter count, peak VRAM, and training throughput, and derive efficiency measures of accuracy per parameter and accuracy per GPU-hour. We will finetune Qwen2.5-1.5B using LoRA ($r = 16$), IA³, and prefix tuning (20 virtual tokens), all in BF16 with matched learning rate and training epochs, and use BF16 full fine-tuning as an upper-bound baseline for accuracy.